

MANUAL DE ARQUITETURA E OPERAÇÃO CORPORATIVA

KCS Hub SaaS (Knowledge-Centered Service AI)

Versão do Documento: 1.0 | Data: Março de 2026

Classificação: Confidencial / Uso Interno Corporativo

1. Introdução e Visão Estratégica

O **KCS Hub** é uma plataforma SaaS projetada para mitigar a perda de capital intelectual e reduzir o *Total Cost of Ownership* (TCO) das operações de TI. A plataforma centraliza resoluções de incidentes, manuais operacionais e scripts de banco de dados (SQL) utilizando a metodologia *Knowledge-Centered Service* potencializada por Inteligência Artificial Generativa.

Objetivos de Negócio:

- Redução do Tempo de Onboarding:** Novos analistas atingem a rampa de produtividade mais rapidamente consultando a base inteligente.
- Redução do Tempo Médio de Atendimento (TMA):** Respostas precisas e instantâneas via Chatbot (RAG Local).
- Prevenção de Riscos:** Validação e explicação de scripts críticos antes da execução em produção.

2. Topologia e Arquitetura do Sistema

O KCS Hub adota uma arquitetura descentralizada, distribuída e sem servidor (*Serverless*), baseada no paradigma **Offline-First**.

- Frontend (Client-Side):** SPA (Single Page Application) construída com HTML5, CSS3 (TailwindCSS) e Vanilla JavaScript (ES6 Modules).
- Banco de Dados Local (Cache):** IndexedDB (gerenciado via [Dexie.js](#)). Garante que a aplicação funcione com latência zero, armazenando os dados no navegador do usuário.
- Backend e Nuvem (BaaS):** Google Firebase.
 - Firestore Database:** Banco de dados NoSQL escalável, atuando como a "Fonte Única da Verdade".
 - Firebase Storage:** Armazenamento em nuvem para upload seguro de imagens e anexos.
 - Firebase Authentication:** Gestão de identidade conectada via Google SSO (Single Sign-On).

- **Motor de Inteligência Artificial:** Google Gemini 2.5 Flash API, atuando como processador de linguagem natural, revisor de texto e orquestrador do Chatbot interno.

3. Segurança da Informação e Conformidade

A segurança da plataforma é garantida por múltiplas camadas de isolamento, alinhadas às melhores práticas corporativas e de proteção de dados (LGPD).

3.1. Arquitetura Multi-tenant (Isolamento Lógico)

A plataforma atende múltiplos clientes (Tenants) em uma única instância de software. O isolamento de dados é garantido diretamente no servidor através de **Firestore Security Rules**. O sistema cruza o `companyId` (Tenant do usuário) com o `companyId` do documento no momento da requisição. Tentativas de acesso trans-tenant são bloqueadas pelo banco de dados antes mesmo de chegarem à aplicação.

3.2. Controle de Acesso Baseado em Identidade (SSO e Whitelist)

O acesso ao sistema é restrito. Não há criação de contas abertas.

1. **SSO Google:** Autenticação unificada garantindo que políticas de senha fortes da empresa sejam aplicadas.
2. **Auto-Provisionamento por Domínio:** O administrador cadastra domínios oficiais (ex: `@empresa.com.br`). Colaboradores com este e-mail têm acesso autorizado automaticamente.
3. **Whitelist de Exceções:** Contas externas (ex: terceirizados, auditores) devem ser previamente convidadas via Painel Administrativo. A ausência de convite ou domínio validado resulta em **Bloqueio Imediato (Acesso Negado)** e destruição do token de sessão.

3.3. Visibilidade Departamental (Segurança em Profundidade)

Os dados são segmentados por `sectorId`. Procedimentos classificados como **"Privados"** contêm criptografia lógica nas regras de nuvem, impedindo que departamentos distintos (ex: TI vs. RH) acessem bases de conhecimento cruzadas.

4. Matriz de Controle de Acesso (RBAC)

O sistema implementa o modelo *Role-Based Access Control* (Controle de Acesso Baseado em Papéis), com quatro perfis de privilégios distintos:

Perfil (role)	Descrição e Permissões

Super Admin	Gerencia a topologia do SaaS. Cria novas empresas (Tenants), define domínios corporativos e tem visão global sobre todos os usuários do sistema.
Admin (Empresa)	Administrador local do Tenant. Pode gerenciar a Whitelist de e-mails, definir os setores dos usuários, atribuir cargos de Analista e revogar acessos na sua própria empresa.
Analista KCS	O "Técnico". Possui permissão de Escrita (write). Cria, edita e deleta procedimentos e scripts SQL. Pode validar artigos para o status "Aprovado".
Usuário Base	O "Consumidor". Possui permissão apenas de Leitura (read). Pode consultar a base, interagir com o Chatbot IA, favoritar artigos, curtir e adicionar comentários.

5. Integração de Inteligência Artificial (Gemini)

A arquitetura do KCS Hub consome a API do Gemini de três formas estratégicas para acelerar a governança:

5.1. RAG Local (Retrieval-Augmented Generation)

O Chatbot (Assistente KCS) não envia perguntas genéricas para a nuvem. Ao receber a dúvida do usuário:

1. O motor de busca semântica (**Fuse.js**) varre a base de dados local (IndexedDB) e isola os 3 documentos/scripts mais relevantes à dúvida.
2. O sistema constrói um *Prompt Mestre* injetando esses 3 documentos de forma oculta.
3. A IA é instruída a responder **estritamente** com base no contexto fornecido, eliminando o risco de alucinações sistêmicas ou fornecimento de informações que não sejam padrão da empresa.

5.2. Estruturação Automática de Rascunhos

Analistas sob pressão tendem a registrar procedimentos de forma desorganizada. O botão "Reescrever IA" coleta todos os campos do formulário (Sintoma, Causa, Ambiente, Solução)

e devolve um documento técnico impecavelmente formatado em *Markdown*, deduzindo automaticamente *Tags* e recomendando a Categoria do sistema adequada.

5.3. Interpretador de Riscos (SQL)

Para prevenir execuções acidentais (ex: um `UPDATE` sem `WHERE`), o módulo de Biblioteca SQL possui a função "Explicar IA". A IA traduz blocos de código complexos para linguagem natural em menos de 2 linhas, garantindo que o analista compreenda o impacto da *query* antes de copiá-la.

6. Resiliência, Sync e Disaster Recovery

A topologia do KCS Hub prevê instabilidades de rede e corrupção de dados através de mecanismos de contingência:

- **Sincronização Offline-First:** Toda gravação é feita primeiro no banco local. A rotina `syncDatabase()` monitora conectividade e envia as alterações ao Firestore de forma assíncrona. Em caso de queda de rede, a operação local não é interrompida.
- **Auto-Cura Lógica (Self-Healing):** Na inicialização (`auth.js`), o sistema verifica a integridade do banco de dados na nuvem. Se encontrar metadados nulos ou `undefined` em perfis de usuários, ele regenera a estrutura com valores *fallback* seguros, evitando telas em branco ou quebras de renderização (UI Freezes).
- **Autonomia de Dependências:** Bibliotecas de terceiros cruciais (como `dexie.js`) são hospedadas internamente na infraestrutura da plataforma, garantindo imunidade contra quedas de CDNs externos ou bloqueios de firewall da rede corporativa.
- **Disaster Recovery (Histórico):** Todo artigo editado salva um *snapshot* imutável no Array de `history`. Em caso de erro humano ou sabotagem, um administrador aciona o botão de **Restauração (Rollback)**, revertendo imediatamente a base para o estado seguro anterior.

7. Logs de Auditoria e Governança

Para rastreabilidade total (Compliance):

1. **Rastreo de Ações:** O sistema anota permanentemente os metadados `createdBy` e `updatedBy` (quem criou e quem modificou), acompanhados de *Timestamps* padronizados.
2. **Sinalização KCS (Flag):** Procedimentos falhos ou desatualizados não são excluídos. O fluxo exige o clique no botão "Reportar Erro", que altera o *Status* do documento para `"review"`, adiciona um banner de aviso visual para outros usuários e registra um log de auditoria via sistema informando qual analista encontrou a falha.

8. Guias de Implantação (Deployment)

O KCS Hub é puramente *Serverless* e compila como arquivos estáticos.

1. **Requisitos de Hospedagem:** Qualquer serviço de hospedagem estática ou CDN estruturado (Google Firebase Hosting, AWS Cloud Storage, Vercel, Cloudflare Pages ou IIS Server corporativo).
2. **Setup de Variáveis (config.js):** As credenciais de produção do Firebase e a API Key do Google Gemini devem ser inseridas no arquivo de configuração principal e, preferencialmente, ofuscadas ou inseridas via variáveis de ambiente no processo de CI/CD (Continuous Integration / Continuous Deployment).
3. **Limpeza de Cache (Deploy First-Run):** Por tratar-se de uma arquitetura baseada em PWA e IndexedDB, após cada novo deploy em produção, recomenda-se orientação aos usuários finais para atualizarem as permissões de cache ou configurarem o *Service Worker* para invalidação automática de versões antigas.

Fim da Documentação Corporativa.